Actionable = Cluster + Contrast?

Rahul Krishna, Tim Menzies
Computer Science, North Carolina State University, USA
{i.m.ralk, tim.menzies}gmail.com

***Abstract*—There are many algorithms for data classification such as C4.5, Naive Bayes, etc. Are these enough for learning actionable analytics? Or should we be supporting another kind of reasoning? This paper explores two approaches for learning minimal, yet effective, changes to software project artifacts.**

***Keywords*—Prediction, planning, instance-based reasoning, model-based reasoning, data mining, software engineering**

I. Introduction

How should we handle "unpopulular" results for data analytics? For example, if a business manager is presented with a data mining result that troubles him (e.g. an estimate of development time that is distressingly long), what advice can we offer him on management actions to *change* that estimate?

This is an important question, and one with much currency in industry. In the summer of 2011 and 2012, one of us (Menzies) spent two months working on-site at Microsoft Redmond, observing data mining analysts. He noted how Microsoft's data scientists discussed their data with business users. One surprising outcome of that study was just how little time was spent inspecting of the output of data miners as compared to another process, which we call *peeking*. In *peeking*, analysts and users spend much time inspecting and discussing small samples of either raw, exemplary, or synthesized project data. Further, very little of those discussions were focused on classification (the addition of a labels to some unlabelled data). Rather, much time was spent in those meetings discussing *what to do next*; i.e. trying to determine what could be altered to better improve some business outcome.

That Microsoft study found two common "peeking" methods. In *data engagement meetings*, users debated the implications of data displayed on a screen. In this way, users engaged with the data and with each other by monitoring each others' queries and checking each others' conclusions.

Another data analysis pattern observed at Microsoft was *cluster + contrast* in which data is reduced to a few clusters. Users are then just shown the delta between those clusters. While contrasting, if feature values are the same in both clusters, then these were pruned from the reports to the user. In this way, very large data sets can be shown on one PowerPoint slide. Note that *cluster+contrast* is a tool that can be usefully employed within *data engagement meetings*.

Cluster+contrast and engagement meetings are common practices at Microsoft. Yet these methods had never been rigorously studied or certified. For both those reasons, we reflected over those tools to discover and analyze their underlying process. The result was BIC [1]: a tool that combines (a) feature selection; (b) centroid generation from clusters; (c) contrast methods between centroids. While method (a) is widely used (e.g. [2]), to the best of our knowledge, this combination of (abc) has not been thoroughly explored before.

This paper assesses two methods for "peeking": a model-based method called XTREE and an instance-based method called BIC. XTREE was first developed by Lekkalapudi and Menzies to explain results from multi-objective optimizers [3]. This paper is the first to apply XTREE to defect prediction and contrast set learning. Also, that prior work evaluated XTREE against multi-objective optimizers and not instance-based methods.

This paper uses three criteria to assess the value of BIC and XTREE for learning actionable analytics:

1) A planning system needs to be *effective*; i.e. if its recommendations are applied then some statistically significant change should be observed. To predict the number of defects in a data set before and after applying our changes, we build a prediction system (built by data mining; specifically: Random Forest). Note that this predictor was built from some hold-out data (i.e. from different data than that used to build the predictor).
2) Any conclusion made to a business user must be understandable; it must generate *succinct* changes.
3) Recommendations should be *stable*; i.e. they shouldn't widely vary due to minor changes in the data. Hence, in our experiments, we will add a little randomness to our analysis then report results across 40 repeated runs.

We will find that, in terms of *effectiveness*, *succinctness*, and *stability* XTREE is better than BIC.

II. How to Cluster

Recent results suggest we are free to select from a wide range of clustering methods. Ganesan [4] explored different clustering methods for SE data using effort and defect data from the PROMISE repository [5]. That study explored K-Means, mini-batch-K-Means, DBscan, EM, Ward, and the WHERE algorithm discussed below. Clusters were assessed via the performance of prediction learned from the clusters by Random Forests (for defect prediction) and M5prime or linear regression (for effect estimation). Ganesan found that the details of the clustering method were less important than ensuring that a large number of clusters were generated.

Accordingly, we select a clustering method that generates $\sqrt{N}$ clusters from N examples, that runs fasts, which has shown promise in prior work [6], and which ignores spurious dimensions (the last item is important since, in our experience, much SE data is "noisey"; i.e. contains signals not associated

Step1: Top down clustering using WHERE
The data is recursively divided in clusters using WHERE as follows:

- Find two distance cases, X, Y by picking any case W at random, then setting X to its most distant case, then setting Y to the case most distant from X (this requires only $O(2N)$ comparisons of N cases).
- Project each case Z onto a `Slope` that runs between X, Y using the cosine rule.
- Split the data at the median X value of all cases and recurses on each half (stopping when one half has less than $\sqrt{N}$ of the original population).

Step2: Distinguish between clusters using decision trees
Call each leaf from WHERE a "class". Use an entropy-based decision tree (DT) learner to learn what attributes select for each "class". To limit tree size:

- Only use the top $\alpha = 33\%$ of the features, as determined by their information gain [7].
- Only build the trees down to max depth of $\beta = 10$.
- Only build subtrees if it contains at least $N^{\gamma=0.5}$ examples (where N is the size of the training set).

Score DT leaf nodes via the mean score of its majority cluster.

Step3: Generating contrast sets from DT branches

- Find the *current* cluster: take each test instance, run it down to a leaf in the DT tree.
- Find the *desired* cluster:
  - Starting at *current*, ascend the tree $lvl \in \{0, 1, 2...\}$ levels;
  - Identify *sibling* clusters; i.e. leaf clusters that can be reached from level lvl that are not *current*
  - Using the *score* defined above, find the *better* siblings; i.e. those with a *score* less than $\epsilon = 0.5$ times the mean score of *current*. If none found, then repeat for $lvl += 1$
  - Return the *closest* better sibling where distance is measured between the mean centroids of that sibling and *current*
- Find the *delta*; i.e. the set difference between conditions in the DT branch to *desired* and *current*. To find that delta:
  - For discrete attributes, return the value from *desired*.
  - For numerics, return the numeric difference.
  - For numerics into ranges, return a random number selected from the low and high boundaries of the that range.

Fig. 1: XTREE. Controlled by the parameters $\{\alpha, \beta, \gamma, \delta, \epsilon\}$ (set via engineering judgement).

with the target variable). For the rest of this article, we use the top-down bi-clustering method described in Figure 1 which recursively divides the data in two using a dimension that captures the greatest variability in the data.

III. HOW TO CONTRAST

BIC uses the WHERE clustering method shown as Step1 in Figure 1. XTREE on the other hand categorizes the data based on a decision tree algorithm. Thus, the algorithms differ in how they report the contrast between the clusters.

A. XTREE

XTREE starts by executing a decision tree algorithm to find what attributes to select for different clusters. After that, to learn recommendations for a business user, it determines (1) what *current* branch a test case is in?; (2) what *desired* branch would the test case have to move to?; (3) what is the delta between those branches of the tree? To answer the last question, we report the delta in the branches of the decision tree that lead to *current* and *desired*. For full details, see Figure 1.

As with any tree structure, the size and depth of the XTREE has a profound impact on its performance. Our initial motivation was that the trees could serve as a medium for experts to identify and explore solution spaces that were local to the problem. The size of the decision tree, if too large, jeopardizes the readability of the solutions by increasing the complexity. We have therefore made efforts to reduce the size of the tree using a pruning method that prunes away irrelevant branches that do not contribute better solutions (see step2 of Figure 1). The smaller trees are much easier to understand.

B. BIC: an Instance-based Approach

XTREE is the latest step in our attempts to combine data mining with planning. All our previous steps were *instance-based methods* that reason directly from the loaded training data without first summarizing that data into a model. The opposite to an instance-based method is one that builds a model from the data, then reasons across that model (e.g. as done by XTREE, above).

Our previous experiment with instance-based planning resulted in W2 [8]. This was an instance-based planner that reflected over the delta of raw attributes (without any initial clustering). However, it frequently suffered from an *optimization failure*: when its plans were applied, performance improved in only $\frac{1}{3}$rd of test cases.

We conjectured that W2 was failing since it could not generalize over sufficient data. Hence, our next experiment was IDEA [9] which first clustered the data with WHERE (see step1 of Figure 1). IDEA made the mistake of summarizing the learned clusters via their median centroid (we now know that to be a mistake since, on experimentation, we found IDEA's optimization failures to be as frequent as those of W2).

The instance-based learner used in this paper is called BIC. During training, BIC (a) clusters the training data with WHERE; (b) assigns a unique id to each cluster; then (c) finds the median centroid of each cluster. Next, working in a random order, BIC labels each cluster with id of the nearest cluster (that has yet to be labelled). This results are *partners* of nearby clusters. After that, BIC draws a *slope* between the centroids of each partner then adds a direction to that *slope* such that "up hill" now points to the centroid of the cluster with a score that is comparatively better than its partner.

When using BIC on test data, each test instance finds its nearest slope then follows the slope "up hill" to find the best centroid in its local region. The delta between that test instance and the best centroid is then BIC's recommendation on how to improve that test instance.

IV. Experiments

A. Data

For our initial experimentation, we studied two sample projects from the Jureczko object-oriented static code data sets [10]: Poi and Lucene [5]. The data is a collection of 20 OO measures and a dependent attribute (number of defects reported to a post-release bug-tracking system).

B. Analysis Methods

Recall from earlier in this paper, we proposed assessing instance-based planner via their *effectiveness*, *stability*, and *succinctness*.

a) Effectiveness: All our results are presented as percent improvement, denoted by $R = (1 - \frac{after}{before}) \times 100\%$. This represents the improvement obtained by applying the recommendations (so larger R indicates larger improvements).

b) Stability: We introduce some small randomness into our recommender systems, then re-run our experiments twenty times (the value of 20 was chosen as the minimum use sample size via the central limit theorem). Our randomness comes from WHERE's initial stochastic selection of instances (recall step1, discussed above) and from the random partner discovery procedure of BIC.

c) Succinctness: We count the frequency attribute changes recommended by XTREE and BIC.

To test if the distributions generated by the above are truly different, we used the the Scott-Knott procedure recommended by Mittas & Angelis in their 2013 IEEE TSE paper [11]. The samples for the Scott-Knott are obtained by repeating our experiments 40 times. With Scott-Knott, when testing for significant differences between samples, we used an Efron & Tibshirani bootstrap procedure [12, p220-223] augmented with an A12 effect size test (and distributions were only ranked different if that difference was recognized by both bootstrapping and the effect size test).

C. Results

1) Effectiveness: The results of figure 2 shows that for our test data, XTREE is more effective than BIC (evidence: the median improvements are larger in XTREE; the Scott-Knott ranks shown in column one assigns a better rank to XTREE than it does to BIC). To further compare these methods, we have to look beyond effectiveness towards stability and succinctness.

2) Succinctness: Figure 3 shows the % change in features proposed by XTREE and instance-based learning. Note that XTREE also fares much better in terms of succinctness since the XTREE propose changes to *far fewer attributes* than instance-based methods. This effect can be better noticed in Figure 4 which shows the mean change across all features. Notice again that XTREE suggests very few changes compared to BIC.

Poi

Rank	Treatment	Median	IQR
1	XTREE	56	13
2	BIC	42	10

Lucene

Rank	Treatment	Median	IQR
1	XTREE	60	11
2	BIC	37	2

Fig. 2: Performance scores on sample projects from the Jurezcko data sets. The median and IQR (inter-quartile range) columns show the 50th and (75th-25th) percentiles.

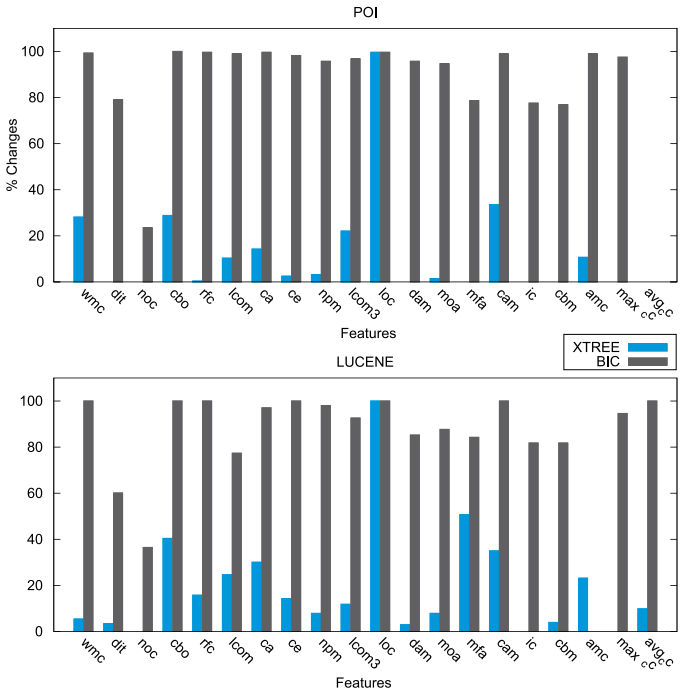

Fig. 3: Performance Comparison. X-axis lists various CK metrics; e.g. loc (lines of code), ce (how many other classes are used by a specific class), cbo (class coupling, measured by number of methods accessing the services of another class). The vertical bars indicate the percent frequency for how often certain feature was changed by a plan.

		XTREE	BIC
Jureczko	Lucene	19%	86%
data	Poi	13%	79%
	mean:	*16%*	*82.5%*

Fig. 4: Average number of features whose values are changed by a planner.

3) Stability: Figure 3 also shows that the above succinctness result is stable across our 40 repeated runs. Further, comparing the sum of the size of all the changes proposed in the top and bottom row of Figure 3, we can see that the recommendations of XTREE is far less varied than those of instance-based contrast methods.

V. Next Steps

This work has primarily been an exploratory study to investigate the efficacy of XTREE as opposed other instance based learning algorithms. Our results provides a natural guide to future research and has encouraged us to explore several avenues of future research. Some of them are discussed below.

a) More data: Tentatively, we can say that the patterns of Figure 3 hold for other data sets from other sources (specifically: model-based methods are as effective as instance-based methods but offer more succinct and stable conclusions). Those results are preliminary and need to better analyzed and documented.

b) Feasibility: Another direction of research we are currently undertaking is a *feasibility study*. With this we would like to understand the nature of changes suggested by XTREE. Specifically: are XTREE's proposed changes small enough to be easily implemented on software projects? Also, are XTREE's proposed changes large enough to be feasible? Here we are asking if users can sufficient fine-grained control to make the small changes proposed in Figure 3.

c) Validation: This work has made use of Random Forest, learned on a hold-out training data, to measure the quality of plans obtained using XTREE. It may be useful to assess the validity of these predictors. This can be achieved in one of many ways such as applying XTREE to to more domains and to domains where exists some "ground truth" for the predictors.

d) Tuning: XTREE was built with several parameters, which were determined after some experimentation. These parameters, $\{\alpha, \beta, \gamma, \delta, \epsilon\}$, used in Figure 1 are excellent candidates for automated tuning.

e) Generality: We have compared two methods for this process and found that a model-based method (XTREE) was effective as an instance-based method (BIC), but the former generated more succinct and stable recommendations. We conjecture that this conclusion holds more generally for other instance-based and model-based methods– and that conjecture requires further testing.

VI. Conclusion

Based on our meeting with Microsoft, we were motivated to explore tools that support *data engagement meetings*: in particular, the process called *peeking* which looks at *cluster+contrast*.

In summary, XTREE shows promise and it will take much further research to establish this technique as a generic tool that can be employed to determine what, and by how much, attributes needs to be altered in order to better improve a business outcome. Thus, for the purposes of "peeking" we recommend XTREE over instance-based methods.

Acknowledgments

The work has partially funded by a National Science Foundation CISE CCF award #1506586.

References

[1] Rahul Krishna, Tim Menzies, Xipeng Shen, and Andrian Marcus. Learning Actionable Analytics (with applications for reducing defects and reducing runtimes). *Submitted to ASE '15, Lincoln, Nebraska*, 2015.

[2] Tim Menzies, Zach Milton, Burak Turhan, Bojan Cukic, Yue Jiang, and Aye Bener. Defect prediction from static code features: Current results, limitations, new approaches. *Automated Software Engineering*, 17(4):375–407, 2010.

[3] Naveen Lekkalapudi. Cross trees: Visualizing estimations using decision trees. Master's thesis, Computer Science, West Virginia University, 2014.

[4] Divya Ganesan. Exploring essential content of defect prediction and effort estimation through data reduction. Master's thesis, Computer Science, West Virginia University, 2014.

[5] The Promise Repository of Empirical Software Engineering Data, 2015. http://openscience.us/repo. North Carolina State University, Department of Computer Science.

[6] Tim Menzies, Andrew Butcher, David Cok, Andrian Marcus, Lucas Layman, Forrest Shull, Burak Turhan, and Thomas Zimmermann. Local versus global lessons for defect prediction and effort estimation. *IEEE Transactions on Software Engineering*, 39(6):822–834, 2013.

[7] KB Irani and UM Fayyad. Multi-lnterval Discretization of Continuous-Valued Attributes for Classification learning. *Proceedings of the National Academy of Sciences of the United States of America*, pages 1022–1027, 1993.

[8] T. Menzies, A. Brady, J. Keung, J. Hihn, S. Williams, O. El-Rawas, P. Green, and B. Boehm. Learning project management decisions: A case study with case-based reasoning versus data farming. *Software Engineering, IEEE Transactions on*, 39(12):1698–1713, Dec 2013.

[9] R Borges and T Menzies. Learning to Change Projects. In *Proceedings of PROMISE'12, Lund, Sweden*, 2012.

[10] Marian Jureczko and Lech Madeyski. Towards identifying software project clusters with regard to defect prediction. In *Proceedings of the 6th International Conference on Predictive Models in Software Engineering*, PROMISE '10, pages 9:1—-9:10. ACM, 2010.

[11] Nikolaos Mittas and Lefteris Angelis. Ranking and clustering software cost estimation models through a multiple comparisons algorithm. *IEEE Trans. Software Eng.*, 39(4):537–551, 2013.

[12] Bradley Efron and Robert J Tibshirani. *An introduction to the bootstrap*. Mono. Stat. Appl. Probab. Chapman and Hall, London, 1993.